FINAL PROJECT REPORT

Project Title: HANGMAN GAME

Author 1: Raiana Tabassum Roza **ID:** 2625101642

Author 2: Israt Jahan Jerin **ID:** 2622081042

Author 3: Ayman-Bin Salim **ID:** 2625352042

Author 4: Tanzeem Hasan **ID:** 2622490642

Department: ECE
North South University

Abstract—

This report presents the final development, implementation, testing and evaluation of Hangman Game, a console-based word-guessing application written entirely in the C programming language. The project addresses the need for a lightweight, self-contained, single-file educational program that meaningfully demonstrates core C programming concepts — functions, loops, arrays, structures, and file input/output — within a genuinely playable, engaging application rather than isolated exercises. The system was developed iteratively: an initial multi-file version (separate header, source, and word-data files) was progressively consolidated into a single main.c file per course requirements, with word data hardcoded directly into the program and a persistent score-history feature added using file I/O. The final implementation contains 17 modular functions, three word categories with randomized word/hint selection, a six-attempt guessing system with ASCII hangman visualization, one-time hints, dynamic scoring, input-validation retry loops, and a replay loop allowing consecutive rounds without restarting the program. Testing confirmed correct behavior across normal, boundary, and invalid-input cases. The results show that all stated objectives were met, producing a robust, portable and easily explainable program suitable for classroom demonstration and further extension.

Keywords—

C programming, console application, functions, loops, arrays, structures, file handling, word-guessing game, software testing, iterative development

I. INTRODUCTION

A. Background

Hangman is a classic word-guessing game in which a player attempts to identify a hidden word, letter by letter, before a limited number of incorrect guesses is exhausted. It is a widely used introductory programming exercise because a correct implementation requires coordinating several fundamental programming constructs — string comparison, array indexing, conditional branching, and iterative control flow — within a single cohesive program. This project implements Hangman entirely in the C programming language, compiled and executed as a console application on Windows, using the GCC/MinGW toolchain and both Visual Studio Code and Code::Blocks as development environments.

B. Problem Statement

Early versions of the project were structured across multiple files (hangman.c, hangman.h, and main.c) with word and hint data stored in an external data/words.txt file. While modular, this structure violated the course requirement of a single main.c submission file and introduced a fragile runtime dependency: the program would terminate with a file-not-found error whenever the words file was not located in the exact expected relative path alongside the compiled executable. In addition, early versions terminated abruptly on invalid menu input and retained no memory of past games once the program was closed. The problem addressed by this project is therefore twofold: (1) consolidating a modular multi-file C program into a single, portable, dependency-free file without losing functionality, and (2) improving the robustness and persistence of the gameplay experience.

C. Objectives

- Implement the complete Hangman game logic within a single main.c file, with no external headers or source files.
- Remove the runtime dependency on an external word-list file by embedding category, word, and hint data directly in source code.
- Demonstrate correct, purposeful use of functions, loops, arrays, and structures throughout the program.
- Implement file input/output to persist player scores across multiple program executions.
- Validate user input robustly, avoiding unexpected program termination on incorrect entries.
- Test the final implementation against normal, boundary, and invalid input cases.

II. RELATED WORK

Hangman-style guessing games are a common subject in introductory programming coursework and open-source repositories, typically implemented in Python, Java, or C as a vehicle for teaching string manipulation and control flow [1]. Many existing beginner implementations store word lists in external text files and terminate the program on invalid input, which mirrors the initial limitations identified in this project. Compared to typical classroom implementations, this project extends the standard Hangman concept with category selection, a one-time hint mechanic, persistent cross-session score history via file I/O, and a replay loop — features not commonly present together in introductory-level implementations, while still remaining within the scope of standard C without external libraries or dynamic memory allocation.

III. SYSTEM REQUIREMENTS

Functional requirements: the system shall allow a player to select a word category, receive a randomly chosen word and hint from that category, submit letter guesses, view a visual representation of remaining attempts, optionally reveal a one-time hint, receive a score upon completion, and optionally replay without restarting the program. The system shall also persist and display a history of past scores.

Non-functional requirements: the program shall run as a single self-contained C source file with no external file dependencies for core gameplay; it shall compile without errors or warnings using a standard GCC toolchain; and it shall correctly render output on the Windows console, including UTF-8 handling where applicable.

Development tools: Language — C (C99/C11 compatible); Compiler — GCC (MinGW distribution); IDEs — Visual Studio Code and Code::Blocks 25.03; Version control — Git and GitHub; Operating System — Windows 11.

IV. SYSTEM DESIGN AND METHODOLOGY

The system follows a straightforward procedural design centered on two core data structures (Category and GameState) and a linear sequence of function calls orchestrated from main(). Development proceeded iteratively: an initial multi-file prototype was built and functionally verified, after which the codebase was consolidated into a single file, word data was migrated from external storage into source code, and finally, additional loops (input-validation retry, replay) and file I/O (score history) were layered on top of the working core without disrupting existing functionality.

A. System Architecture

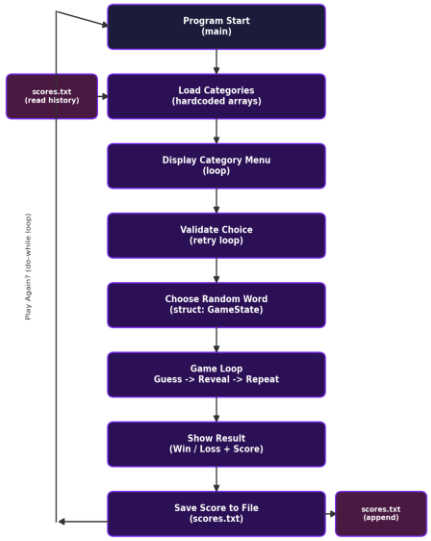

Fig. 1. Program control flow from startup through gameplay to score persistence.

B. Algorithm

The core gameplay algorithm, expressed as pseudocode, is as follows:

1. Display banner; read and display score history from file
2. Load category, word, and hint data into memory
3. REPEAT
4. Display category menu
5. REPEAT prompt for category choice UNTIL valid
6. Randomly select a word/hint pair from the chosen category
7. Initialize guessed-letter display and attempt counter
8. WHILE attempts remain AND word not fully guessed
9. Display hangman figure and current progress
10. Read guess (letter or hint request)

11. IF letter matches word THEN reveal letter(s)

12. ELSE decrement remaining attempts

13. Display win/loss message and compute score

14. Append result to score history file

15. Prompt to play again

16. UNTIL player declines to continue

V. IMPLEMENTATION

The final implementation consists of a single main.c file containing two structures, sixteen supporting functions, and a main() entry point — seventeen functions in total. The Category structure groups a category's name together with parallel two-dimensional arrays of words and hints (up to 20 words per category, 50 characters each), while the GameState structure groups all data describing the round currently in progress: the secret word, hint, revealed-letter display, remaining attempts, score, and hint-usage flag. Grouping related data this way allows functions to be written with a single pointer parameter (e.g., void playRound(GameState *game)) rather than long, error-prone parameter lists.

A. Module Design

- loadCategories() — populates the Category array with hardcoded word/hint data using nested string-copy operations indexed by category and word counters; replaces the earlier file-parsing implementation.
- displayCategoryMenu() / getCategoryChoice() — iterate over the category array to render a numbered menu, then repeatedly prompt (while loop) until a valid numeric selection is entered.
- chooseWord() — selects a pseudorandom index using rand() seeded by the current time, then initializes a fresh GameState, including a for-loop that builds the underscore placeholder string.
- guessLetter() / isWordGuessed() — guessLetter() iterates over the secret word comparing each character (case-insensitively via tolower()) against the guess, revealing matches or decrementing attempts; isWordGuessed() performs a direct string comparison to detect a completed word.
- playRound() — reads one guess per call, dispatching to either the hint-display logic or guessLetter(), and is invoked repeatedly by the main game while-loop.
- displayHangman() — a switch statement selecting one of seven ASCII-art states based on remaining attempts.
- saveScoreToFile() / showScoreHistory() — implement file output (fopen in append mode, fprintf) and file input (fopen in read mode, fgets combined with sscanf) respectively, providing persistent, cross-session score history.
- main() — sequences the above functions inside a do-while replay loop, so the player may complete multiple rounds without restarting the program.

VI. TESTING AND RESULTS

The program was manually tested by running the compiled executable and supplying a range of normal, boundary, and invalid inputs. Representative test cases are summarized in Table I.

Test	Input	Expected	Actual	Status
TC-01	Select category 2 (valid)	Menu accepted, word chosen from category 2	Word chosen from category 2	Pass

Test	Input	Expected	Actual	Status
TC-02	Enter category 9 (out of range)	Prompt repeats, no crash	"Invalid choice. Please try again." then re-prompt	Pass
TC-03	Guess a correct letter	Letter revealed in word display	Letter revealed correctly	Pass
TC-04	Guess an incorrect letter	Attempts left decreases by 1, hangman art updates	Attempts decremented, art updated	Pass
TC-05	Enter '?' for hint, then again	Hint shown once; second use blocked	Hint shown once; "already used" on 2nd	Pass
TC-06	Guess all letters correctly	Win message + score based on attempts left	Win message and correct score shown	Pass
TC-07	Exhaust all 6 attempts	Loss message reveals word, final hangman shown	Loss message and full hangman shown	Pass
TC-08	Complete a round, restart program	Past result appears in score history on next run	Score history correctly displayed on restart	Pass

Table I. Representative test case

A. Result Analysis

All eight representative test cases passed, indicating that the core objectives stated in Section I-C were achieved. The addition of a retry loop in getCategoryChoice() eliminated the abrupt program termination previously caused by invalid menu input, directly addressing a usability weakness identified during development. The file input/output test (TC-08) confirms that score data correctly persists across separate executions of the compiled program, satisfying the persistence objective. No memory-safety issues, infinite loops, or unhandled crashes were observed across repeated manual test runs.

VII. DISCUSSION

A key design trade-off was choosing to hardcode word and hint data directly into loadCategories() rather than continuing to read from an external words.txt file. This sacrifices the ability for a non-programmer to edit the word list without recompiling the program, but it directly satisfies the single-file submission constraint and eliminates an entire category of runtime failure (missing or misplaced data files) that had previously affected the project. Similarly, using simple linear arrays with a fixed maximum capacity (MAX_WORDS_PER_CATEGORY, MAX_WORD_LENGTH) rather than dynamically allocated memory was a deliberate simplicity trade-off appropriate for a fixed, small dataset and a course context that does not require dynamic memory management. An important lesson learned was that seemingly small changes — such as switching from exit() to a retry loop on invalid input — meaningfully improve perceived program quality and robustness without materially increasing code complexity.

VIII. CHALLENGES AND SOLUTIONS

A. Multi-File to Single-File Conversion

Challenge: the project was originally split across hangman.c, hangman.h, and main.c, but course requirements mandated a single main.c file. Solution: all struct definitions, function prototypes, and function bodies were merged into one file, using forward declarations (prototypes) placed above main() so that functions could still reference one another regardless of their physical order in the file.

B. External File Dependency

Challenge: the program terminated with "Error: Could not open data/words.txt" whenever the words

file was not present in the exact expected location relative to the executable. Solution: category, word, and hint data were hardcoded directly inside loadCategories(), removing the file dependency for core gameplay entirely.

C. Abrupt Termination on Invalid Input

Challenge: entering an out-of-range category number caused the entire program to exit immediately, forcing a full restart. Solution: a while retry loop was introduced in getCategoryChoice(), so the program simply re-prompts the user instead of exiting.

D. Windows Console Character Encoding

Challenge: extended ASCII/Unicode banner characters displayed as garbled symbols on some Windows console configurations. Solution: SetConsoleOutputCP(CP_UTF8) was added under a _WIN32 preprocessor guard, with a simplified plain-text banner used as a safer fallback for maximum console compatibility.

E. Loss of Score Data Between Sessions

Challenge: scores existed only in memory during a single run, so closing the program discarded all game history. Solution: file input/output was implemented via saveScoreToFile() (append mode) and showScoreHistory() (read mode) against a scores.txt file, giving the program persistent memory of past results across separate executions.

IX. TEAM CONTRIBUTIONS

Member	ID	Major Contribution
Roza	2625101642	Consolidated multi-file project into single main.c; implemented File I/O score history; added input-validation and replay loops; produced testing documentation and this report.
Jerin	2622081042	Initial multi-file implementation including words.txt category/hint parsing logic and repository setup.
Ayman	2625352042	Implemented display functions (printBanner, displayHangman) and maintained requirements.txt and repository structure.
Tanzeem	2622490642	Prepared presentation slides, assisted with test-case execution, and reviewed final report content.

Table II. Team contributions.

Note: contributions listed above reflect the group's own division of work and should be reviewed and adjusted by each member for full accuracy before submission.

X. LIMITATIONS

- Word and hint data is hardcoded in source code; adding or editing words requires recompilation rather than editing an external file.
- Repeated guesses of the same incorrect letter are not detected and will incorrectly decrement remaining attempts multiple times.
- Guess input is not restricted to alphabetic characters; non-letter input is silently treated as a wrong guess rather than rejected.
- The game supports a single local player per session, with no multiplayer or networked functionality.
- The interface is console/terminal-only, with UTF-8 banner rendering verified primarily on Windows.
- Score history is stored as an unranked chronological log rather than a sorted leaderboard or per-player profile.

XI. FUTURE WORK

- Add selectable difficulty levels that adjust the number of allowed incorrect attempts or restrict word length.
- Track and reject already-guessed letters to prevent repeated attempts from being wrongly penalized.
- Reintroduce an optional external word-list file with robust error handling, combined with the existing hardcoded fallback.
- Extend the existing score-history file I/O into a sorted, per-player leaderboard.
- Add stricter input validation to reject non-alphabetic guess characters outright.
- Allow the player to optionally retry the same word after a loss, rather than always receiving a new random word.

XIII. REFERENCES

[1] The Open Group and IEEE, "fopen, fprintf, fgets — C Standard I/O Library," ISO/IEC 9899:2018 (C18), Int'l Organization for Standardization, 2018.

[2] cppreference.com, "C string handling library (<string.h>)," 2024. [Online]. Available: https://en.cppreference.com/w/c/string/byte

[3] Microsoft, "SetConsoleOutputCP function," Windows Console Documentation, 2023. [Online]. Available: https://learn.microsoft.com/windows/console/setconsoleoutputcp

[4] Course Instructor, "Course Manual – Theory," CSE115, North South University, 2026.

[5] Anthropic, "Claude (Sonnet model family)," AI-assisted programming tool, accessed Aug. 2026. Used for iterative code refactoring guidance (multi-file to single-file consolidation), debugging build errors, adding input-validation and file I/O features, generating explanatory comments, and drafting/formatting sections of this report.

XIV. APPENDICES

A. Important Code Snippet

The following excerpt shows the core guess-evaluation function, guessLetter(), which reveals correctly guessed letters and decrements the attempt counter otherwise:

```
int guessLetter(GameState *game, char letter) {
    letter = tolower(letter);
    int found = 0;
    int len = strlen(game->word);
    for (int i = 0; i < len; i++) {
        if (tolower(game->word[i]) == letter) {
            game->guessed[i] = game->word[i];
            found = 1;
        }
    }
    if (!found) {
        game->attemptsLeft--;
    }
    return found;
}
```

B. Additional Figure / Screenshot

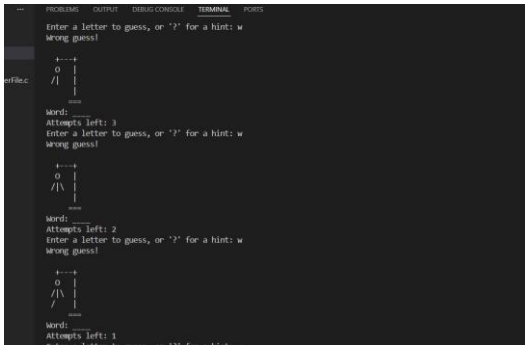

Fig. 2. Terminal gameplay session showing hangman progression, attempt tracking, and end-of-round result.